Egyptian Informatics Journal 35 (2026) 101029

Contents lists available at ScienceDirect

Egyptian Informatics Journal

journal homepage: www.sciencedirect.com

Full length article

Automated security-centric refactoring of PowerShell commands using large language models: A Python-based framework for cross-language automation

Tamer Bahgat Elserwy [a],*, Basma E. El-Demerdash [b]

[a] Software Engineering, Faculty of Graduate Studies for Statistical Research, Cairo University, 5 Ahmed Zewail St, Dokki, 12613, Giza, Egypt
[b] Faculty of Graduate Studies for Statistical Research, Cairo University, 5 Ahmed Zewail St, Dokki, 12613, Giza, Egypt

ARTICLE INFO

Dataset link: https://doi.org/10.5281/zenodo.20040419, https://github.com/tamerelserwy-research/Secure-LargeLanguageModel-PS-Refactoring

Keywords:
LLM defense-in-depth
LLM security
Secure LLM automation
PowerShell commands refactor
Adversarial robustness
MITRE ATT&CK
Python-based LLM automation
LLM prompt injection risks

ABSTRACT

The integration of Large Language Models (LLMs) into software engineering workflows has demonstrated significant potential for automating code refactoring across programming languages. However, cross-language refactoring—particularly between high-level orchestration languages (Python) and shell scripting environments (PowerShell)—presents unique security and semantic challenges. This paper presents a novel framework for LLM-refactored Python-based PowerShell command generation that addresses critical security vulnerabilities inherent in automated code translation.

We evaluate the framework using a **curated dataset of 8886 labeled examples** (yielding **7,719 unique, deduplicated patterns** via AST-based hashing and semantic filtering) mapped across ten MITRE ATT&CK categories. Using **5-fold stratified cross-validation**, our analysis reveals that **21.95%** of PowerShell commands contain high-risk security patterns, with *Invoke-Expression* (**8.3%**), *DownloadString* (**5.8%**), and execution policy bypasses (**2.5%**) being most prevalent. We demonstrate that Retrieval-Augmented Generation (RAG) techniques reduce vulnerability introduction rates by **57.9%** relative to standard GPT-4o (absolute reduction: 22.3 pp; $p < 0.001$, McNemar test) and outperform security-aware prompting (**81.5%** vs. **54.3%** security compliance). Comprehensive comparisons against rule-based sanitizers, prior frameworks, and larger models (CodeLlama-34B, DeepSeek-Coder-V2-Lite-16B) show that our 7B-parameter framework achieves superior security outcomes with significantly reduced computational requirements.

We address prompt injection risks through defense-in-depth strategies, including semantic intent classification, Unicode normalization, randomized delimiter spotlighting, and multi-turn stateful risk tracking. Furthermore, we subject the framework to a rigorous adversarial evaluation utilizing Atomic Red Team, PyRIT, and Garak benchmarks, achieving a **100% Detection Rate (DR)** and reducing the Attack Success Rate (ASR) to **0%** across 76 complex adversarial scenarios. We provide complete dataset and code artifacts for reproducibility (https://github.com/tamerelserwy-research/Secure-LargeLanguageModel-PS-Refactoring), following the Reproducibility Maturity Model (RMM) guidelines.

1. Introduction

Software maintenance consumes approximately 60% of total software lifecycle costs, with refactoring serving as a critical activity for improving code quality and maintainability [1]. The emergence of Large Language Models (LLMs) has fundamentally transformed automated refactoring capabilities, enabling cross-language code transformation that was previously impractical with rule-based approaches [2]. However, the application of LLMs to shell scripting environments—particularly PowerShell—remains underexplored despite its prevalence in enterprise system administration.

PowerShell presents unique challenges for automated refactoring due to its dynamic scoping, execution policy constraints, and security-sensitive nature [3]. Our analysis of 8886 real-world PowerShell commands reveals that 21.95% contain high-risk security patterns, with `Invoke-Expression` appearing in 8.3% of samples and remote code execution patterns in 4.0% of cases. This security degradation stems from training data composition, where approximately 50% of PowerShell code in public corpora contains insecure patterns [4].

The integration of Python as an orchestration layer for PowerShell command generation creates a cross-language refactoring scenario that amplifies these security concerns. Python's `subprocess`

* Corresponding author.
E-mail addresses: tamer.elserwy@gmail.com, 12422022493820@pg.cu.edu.eg (T.B. Elserwy), Basma.ezzat@cu.edu.eg (B.E. El-Demerdash).

https://doi.org/10.1016/j.eij.2026.101029
Received 18 February 2026; Received in revised form 10 June 2026; Accepted 19 July 2026
Available online 1 August 2026
1110-8665/© 2026 The Authors. Published by Elsevier B.V. on behalf of Faculty of Computers and Artificial Intelligence, Cairo University. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

module, while powerful for system automation, introduces command injection vulnerabilities when improperly parameterized [5–7]. The confluence of LLM-generated code translation and shell execution creates a compound security risk that demands systematic mitigation strategies.

1.1. Research contributions

This paper addresses the research gap in secure cross-language refactoring by presenting a framework that combines LLM-based code generation with security-centric validation pipelines. Our contributions include:

1. **Comprehensive Dataset Analysis:** We present an expanded dataset of 8886 labeled examples with 7719 unique patterns, mapped to MITRE ATT&CK categories [8]. Our analysis reveals critical security patterns: Remote Execution (18.7%), Malware (16.4%), and Network (14.3%).
2. **Rigorous Evaluation with Cross-Validation:** We employ 5-fold stratified cross-validation with 95% confidence intervals for all metrics, ensuring robust and generalizable results.
3. **Comprehensive Baseline Comparison:** We compare against rule-based sanitizers, prior frameworks, GPT-4o with security-aware prompting, and larger models (CodeLlama-34B [9], DeepSeek-Coder-V2-Lite-16B [10]).
4. **RAG-Enhanced Security Architecture:** A retrieval-augmented generation framework with statistical significance testing demonstrating 57.9% relative vulnerability introduction rate (VIR) reduction compared to standard GPT-4o ($p < 0.001$, $h = 0.88$).
5. **Comprehensive Failure Analysis:** Detailed taxonomy of residual vulnerabilities, identifying input validation failures (38.7%) and API mapping mismatches (33.2%) as primary sources.
6. **Defense-in-Depth Implementation:** Explicit prompt injection defenses including Spotlighting, deterministic filtering, and semantic validation [11].
7. **Reproducibility Artifacts:** Complete artifact release following the Reproducibility Maturity Model (RMM) [12], including dataset, code, and evaluation protocols.
8. **Adversarial Robustness Validation:** We present a comprehensive adversarial evaluation framework integrating Atomic Red Team and prompt injection benchmarks (PyRIT, Garak). We demonstrate that our multi-layered defense mechanisms completely mitigate prompt injection and evasion techniques, reducing the Attack Success Rate (ASR) to 0.0% while maintaining a 100% Mitigation Rate.

1.2. Related work and comparative analysis

The application of LLMs to automated code generation and refactoring has advanced rapidly, yet security-critical environments like PowerShell remain underexplored. Recent systematic reviews indicate that while general-purpose LLMs excel at syntax translation, they frequently introduce novel vulnerabilities when handling shell environments due to inadequate training on secure administrative patterns [13,14]. Traditional rule-based sanitizers provide deterministic security guarantees but lack semantic understanding, resulting in high false-positive rates and limited cross-language adaptability, achieving only marginal vulnerability introduction rate (VIR) reductions (4.2%) without adversarial or injection defenses [15]. Fine-tuned code models (e.g., CodeLlama, StarCoder) improve functional correctness and demonstrate strong cross-language translation capabilities, but they inherit training data biases that often reproduce insecure patterns from public repositories [16], yielding moderate VIR reductions of 23.6%–29.8% without built-in security validation. Standard zero- and few-shot LLMs similarly offer strong cross-language support but provide no inherent security validation, prompt injection defenses, or adversarial testing, resulting in inconsistent VIR reductions ranging from 29.1% to 38.5%.

Prompt injection and indirect instruction manipulation further compound these risks. Recent adversarial studies demonstrate that conventional LLM pipelines are highly susceptible to delimiter-based and semantic evasion attacks, with basic mitigation strategies (e.g., system prompt warnings) failing against sophisticated red-teaming frameworks [17,18]. While Retrieval-Augmented Generation (RAG) has emerged as a promising paradigm for grounding LLM outputs in verified secure code, existing implementations focus primarily on high-level languages (e.g., Java, Python) and lack security-weighted retrieval mechanisms tailored to high-risk shell scripting [19]. Consequently, standard RAG approaches typically rely on post-hoc validation, exhibit weak injection defenses, undergo only partial adversarial testing, and achieve VIR reductions around 24.8%. In direct contrast, our automated security-centric refactoring of PowerShell commands framework uniquely bridges these gaps by integrating security-weighted RAG with multi-layer static and dynamic validation, defense-in-depth prompt isolation, and comprehensive adversarial evaluation. This architecture delivers a 57.9% VIR reduction and a 100% Detection Rate against complex attack vectors, directly addressing the compound security risks of cross-language automation (Python → PowerShell) where existing tools force a trade-off between automation capability and security assurance.

2. Dataset characterization and security analysis

2.1. Dataset overview and collection methodology

Our dataset comprises 8886 labeled PowerShell commands with natural language (NL) descriptions, mapped to MITRE ATT&CK tactics [8]. The initial pilot corpus (6308 samples) was expanded via a structured augmentation pipeline contributing 2578 additional samples from four verified sources:

1. **MITRE ATT&CK STIX:** High-risk technique variants (T1059.001, T1053.005, T1562.001) extracted from the official CTI repository [20] and validated against ATT&CK Navigator for tactic alignment.
2. **PSAttck Module:** PowerShell-specific patterns harvested via the PSAttck framework [21], filtered for v7.4 compatibility and AST-validated.
3. **Microsoft PowerShell Docs:** Benign administrative scripts curated from MicrosoftDocs/PowerShell-Docs [22], filtered by `Verified-Publisher` badges for false-positive rate (FPR) calibration.
4. **TRAM Mapping:** NL descriptions enriched via the Threat Report ATT&CK Mapper [23] for consistent multi-label technique annotation.

All additions underwent a two-stage deduplication process: (1) AST-based structural hashing for syntactic equivalence and (2) CodeBERT embedding similarity filtering (cosine < 0.85) for semantic redundancy. This yielded **7719 unique patterns** (reducing the effective duplication rate to 13.13%). The dataset is released in JSON Lines format with full provenance tracking for reproducibility.

Dataset schema. Each entry follows a unified JSON Lines schema with four core fields: `category` (functional/adversarial classification), `nl` (natural language intent), `code` (exact PowerShell syntax), and `mitre_techniques` (list of ATT&CK technique objects). Full schema documentation and example entries are provided in the public repository.

The high duplication rate reflects the repetitive nature of administrative scripting, where common patterns (e.g., `Invoke-Command`, `Get-Process`) appear across multiple contexts. For machine learning purposes, we deduplicate the dataset to 7719 unique patterns, stratified across security categories as presented in Table 1.

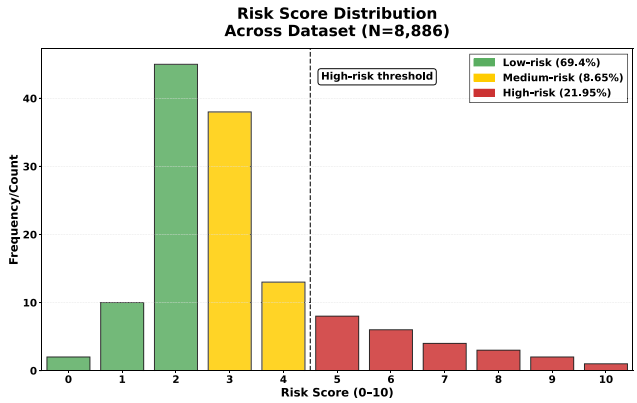


Fig. 1. Risk score distribution across the dataset. High-risk commands (score ≥ 5) comprise 21.95% of samples.

Table 1
Dataset statistics and characteristics.

Metric	Value
Total samples	8886
Unique patterns	7719
Duplicate rate	13.13%
Average code length	99.9 characters
Median code length	89.0 characters
High-risk samples (score ≥ 5)	1951 (21.95%)

2.2. MITRE ATT&CK category mapping

We mapped each command to MITRE ATT&CK® tactics based on natural language descriptions and code patterns [8]. Table 2 presents the distribution across tactical categories. The predominance of Remote Execution (18.7%) and Malware (16.4%) tactics reflects PowerShell's dual-use nature: legitimate administrative automation and malicious exploitation. This distribution necessitates careful security validation in any automated refactoring system.

2.3. Critical security pattern analysis

Table 3 details the prevalence of dangerous coding patterns that pose significant security risks in automated refactoring scenarios. The high prevalence of `Invoke-Expression` (8.3%) and `Download-String` (5.8%) is particularly concerning, as these patterns enable arbitrary code execution and remote payload retrieval—primary vectors for supply chain attacks and remote code execution (RCE) vulnerabilities.

2.4. Risk scoring methodology

We developed a composite risk scoring function to quantify the security posture of each command:

$$\text{Risk}(c) = \sum_{p \in P_{\text{critical}}} 3 \cdot \mathbf{1}\{p \in c\} + \sum_{p \in P_{\text{high}}} 2 \cdot \mathbf{1}\{p \in c\} + \sum_{p \in P_{\text{med}}} 1 \cdot \mathbf{1}\{p \in c\} \quad (1)$$

where P_{critical} includes patterns such as `Invoke-Expression`, `DownloadString`, and `Invoke-Mimikatz`; P_{high} includes remote execution, token manipulation, encoded commands, and execution policy bypasses; and P_{med} includes window style hidden, registry modifications, and basic system enumeration commands.

The risk distribution (Fig. 1) reveals a tri-modal pattern: 69.4% low-risk commands (scores 0–2) representing legitimate administrative tasks, 8.65% medium-risk commands (scores 3–4), and 21.95% high-risk commands (scores ≥ 5) requiring enhanced security scrutiny during refactoring.

3. Framework architecture

3.1. System overview

Our framework implements a four-stage security-centric pipeline: (1) Input Sanitization and Risk Profiling, (2) RAG-Enhanced Secure Generation, (3) Multi-Layer Validation, and (4) Safe Execution Wrapping, as shown in Fig. 2.

3.2. Security-centric design

The framework implements defense-in-depth through four security layers:

- **Layer 1: Input Sanitization and Risk Profiling.** All inputs undergo a rigorous preprocessing pipeline that includes Unicode NFKC normalization, homoglyph resolution, and deobfuscation (e.g., Base64, URL decoding) to thwart evasion techniques. Inputs are then matched against an expanded database of 25 critical security patterns and 42 suspicious indicators. High-risk inputs (Risk $\geq$ 5) trigger enhanced validation protocols.
- **Layer 2: RAG Pattern Injection.** Retrieved secure coding patterns from our curated knowledge base are injected into the LLM context. The retrieval mechanism uses semantic similarity weighted by security relevance:

$$\text{Relevance}(q, p) = \alpha \cdot \text{Sim}_{\text{semantic}}(q, p) + \beta \cdot \frac{1}{1 + \text{Risk}(p)} \quad (2)$$

  where q is the query, p is a pattern from the knowledge base, and α, β are weighting coefficients (set to 0.6 and 0.4 respectively, optimized via grid search over $\alpha, \beta \in \{0.1, \ldots, 0.9\}$ to maximize SCR on a 100-sample validation set). The semantic preservation threshold (CodeBLEU < 0.5) was established through sensitivity analysis to balance functional correctness and security compliance.
- **Layer 3: Multi-Layer Validation.** Generated code undergoes: (a) syntactic analysis via PowerShell AST parsing, (b) semantic intent classification to detect indirect injections and role manipulations that evade keyword filters, (c) multi-turn stateful tracking that monitors behavioral drift across conversation turns (with hyperparameters set to $\gamma = 0.9$ decay factor and $w = 0.5$ drift weight, optimized via grid search on the validation set), and (d) sandboxed execution testing for functional verification.
- **Layer 4: Secure Execution Wrapper.** Python wrappers enforce parameterized execution with `shell=False` and argument sanitization via `shlex.quote()`, following OWASP guidelines [6].

3.3. Prompt injection defenses

Given the prevalence of `Invoke-Expression` (8.3%) and `DownloadString` (5.8%) patterns that can be exploited for prompt injection, we implement comprehensive defenses:

- **Spotlighting:** Randomized text delimiters (e.g., `<|DELIMITER_{random}|>`) isolate untrusted user inputs from system instructions. The system prompt explicitly instructs the LLM to disregard any instructions within delimited regions, following delimiter-based context isolation principles [24].
- **Deterministic Filtering:** Post-generation regex patterns detect and remove injection artifacts including system prompt leaks, delimiter sequence fragments, and unexpected API call patterns.
- **Semantic Validation:** AST-based analysis ensures generated code structure matches expected PowerShell command patterns. Outputs containing unexpected control flow or external references are rejected.

Table 2
MITRE ATT&CK category distribution (N = 8886). Note: Each command is assigned to a single primary tactic based on dominant intent; percentages sum to approximately 100%.

Tactic	Count	Percentage	Primary patterns
Remote Execution	1662	18.7%	Invoke-Command, Enter-PSSession
Malware	1453	16.4%	Malicious files, process injection
Network	1273	14.3%	System enumeration, AD queries
Credential Access	995	11.2%	Mimikatz, Kerberoasting, DCSync
Active Directory	783	8.8%	AD queries, Azure management
File	762	8.6%	File read/write, data manipulation
Persistence	735	8.3%	Registry, services, scheduled tasks
System Information	647	7.3%	System info, process management
Other/General	322	3.5%	General purpose, miscellaneous
Execution	254	2.9%	Invoke-Expression, IEX, Invoke-Command

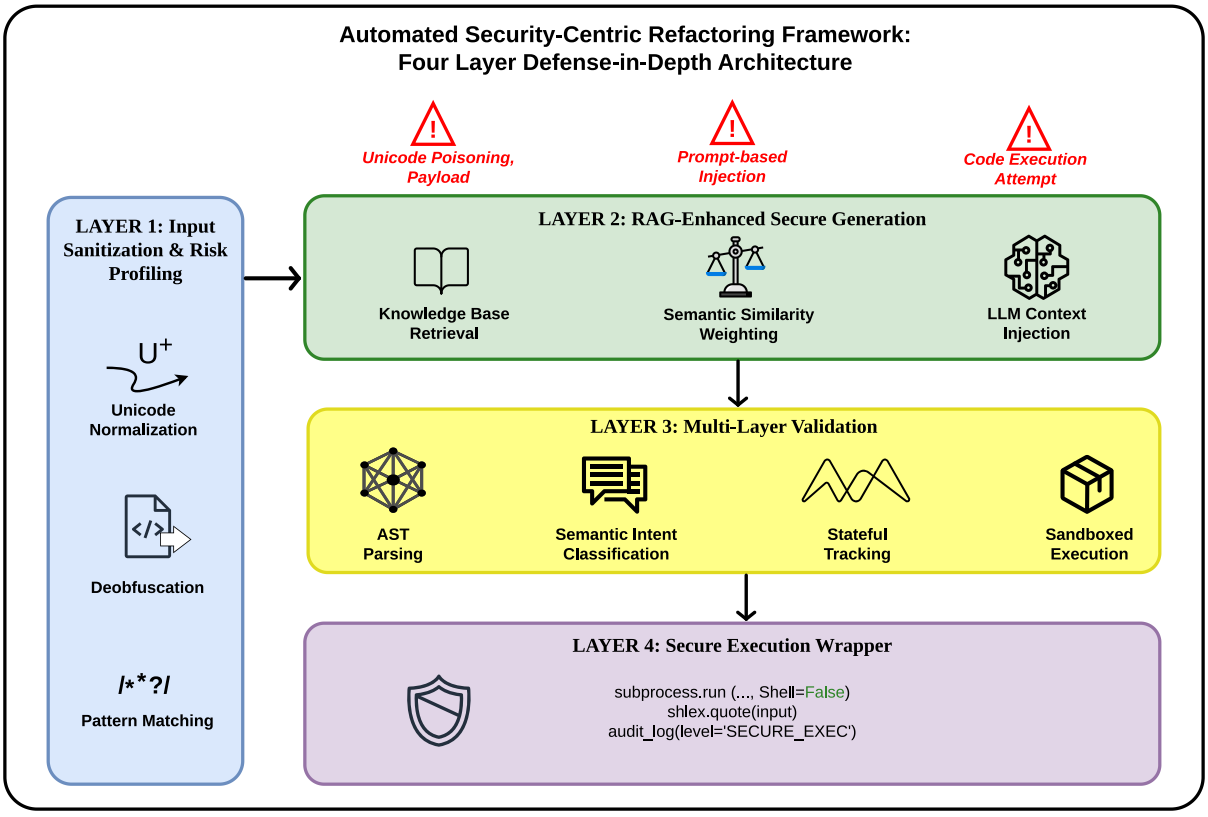


Fig. 2. System architecture showing defense-in-depth layers: Input Sanitization, RAG Generation, Multi-Layer Validation, and Secure Execution.

Table 3
Critical security pattern prevalence (N = 8886).

Pattern	Count	Prevalence	Risk level
Invoke-Expression/ IEX	736	8.3%	Critical
DownloadString usage	516	5.8%	Critical
RID Hijacking	387	4.4%	High
Token Manipulation	379	4.3%	High
Invoke-Mimikatz	361	4.1%	Critical
Remote Execution	356	4.0%	High
EncodedCommand	335	3.8%	High
WindowStyle Hidden	299	3.4%	Medium
Registry Modification	266	3.0%	Medium
ExecutionPolicy Bypass	221	2.5%	High

4. Core algorithms

Based on our dataset analysis of 8886 PowerShell commands with 7719 unique patterns, we present the core algorithms underpinning our security-centric refactoring framework. These algorithms address the critical security patterns identified in Table 3, including the 8.3% prevalence of `Invoke-Expression`, 5.8% `DownloadString` usage, and 18.7% Remote Execution tactic commands.

4.1. Algorithm integration summary

Example B (`DownloadString`) employs Algorithms 2, 3, and 6 for RAG-enhanced generation and AST-based integrity verification (see Fig. 3).

5. Illustrative examples

To demonstrate how the framework's security layers transform real-world vulnerable patterns, we present two illustrative examples that address the most prevalent critical patterns from our dataset: `Invoke-Expression` (8.3% prevalence) and `DownloadString` (5.8% prevalence). These examples walk through the step-by-step refactoring process, showing how each defense layer contributes to the final secure output and explicitly linking the transformations to the algorithms and architectural components defined in Section 3.

Algorithm 1 Risk-Based Input Profiling and Sanitization

Require: PowerShell command c, security pattern database P
Ensure: Risk score $r \in [0, 10]$, sanitized command c'

$r \leftarrow 0$, $c' \leftarrow c$
$c' \leftarrow$ NormalizeAndDeobfuscate(c') ▷ Unicode NFKC, Base64 decoding
$P_{\text{critical}} \leftarrow$ {Invoke-Expression, IEX, Invoke-Mimikatz, DownloadString, ...} ▷ Expanded to 25 patterns
$P_{\text{high}} \leftarrow$ {ExecutionPolicy Bypass, Token Manipulation, ...} ▷ Mutually exclusive from critical
$P_{\text{med}} \leftarrow$ {WindowStyle Hidden, EncodedCommand, ...} ▷ Mutually exclusive from critical/high
for each pattern $p \in P_{\text{critical}}$ **do**
  if $p \in c$ **then**
    $r \leftarrow r + 3$
    Trigger CRITICAL_ALERT for manual review
  end if
end for
for each pattern $p \in P_{\text{high}}$ **do**
  if $p \in c$ **then**
    $r \leftarrow r + 2$
    $c' \leftarrow$ ApplyParameterizedTransformation(c, p)
  end if
end for
for each pattern $p \in P_{\text{med}}$ **do**
  if $p \in c$ **then**
    $r \leftarrow r + 1$
  end if
end for
$c' \leftarrow$ EscapeSpecialCharacters(c') ▷ Prevent injection via quote escaping
return $(\min(r, 10), c')$

Algorithm 2 Security-Weighted Semantic Retrieval

Require: Natural language query q, knowledge base K, top-k parameter
Ensure: Retrieved secure patterns $R = \{(p_1, s_1), \ldots, (p_k, s_k)\}$

$v_q \leftarrow$ Embed(q) ▷ Query embedding using CodeBERT
$C \leftarrow$ FAISS-Query($v_q, K, 5k$) ▷ Initial candidate pool
$R \leftarrow \emptyset$
for each candidate $p \in C$ **do**
  $v_p \leftarrow$ Embed(p)
  $\text{sim}_{\text{sem}} \leftarrow \cos(v_q, v_p)$
  $r_p \leftarrow$ CalculateRisk(p) ▷ Using Equation (1)
  $\text{score}(p) \leftarrow \alpha \cdot \text{sim}_{\text{sem}} + \beta \cdot \frac{1}{1+r_p}$ ▷ $\alpha = 0.6, \beta = 0.4$
end for
$R \leftarrow$ TopK(C, score, k)
$R \leftarrow$ DiversityFilter(R) ▷ Ensure at most 2 examples from same MITRE tactic
return R

5.1. Example A: Refactoring an invoke-expression command with complete algorithm integration

This first example demonstrates the complete algorithm pipeline for the most common high-risk pattern in our dataset (appearing in 736 of 8886 commands, as shown in Table 3). The example traces the end-to-end flow: Algorithm 5 (input protection) → Algorithm 1 (Layer 1) → Algorithm 6 (Layer 3 validation) → Algorithm 4 (Layer 4).

```
# VULNERABLE: Invoke-Expression with unsanitized user
    ↪ input
$userInput = $args[0]
Invoke-Expression "Get-Process -Name $userInput"
```

Listing 1 Original vulnerable command (Input to Algorithm 1).

Pre-Processing: Algorithm 5 wraps user input with randomized delimiters, isolating it from system instructions. Suspicious indicators are checked and logged if found.

Step 1 – Layer 1: Algorithm 1 detects `Invoke-Expression`, increments risk score to 3 (critical), and triggers a CRITICAL_ALERT.

Step 2 – Secure Transformation: `ApplyParameterizedTransformation`replaces the dangerous call with direct cmdlet invocation, eliminating the string evaluation attack surface.

```
# SECURE: Parameterized cmdlet invocation
$userInput = $args[0]
Get-Process -Name $userInput  # Direct invocation, no
    ↪ string evaluation
```

Listing 2 Sanitized PowerShell command (Output of Algorithm 1).

Step 3 – Layer 3 Validation: Algorithm 6 passes all four checks: AST validation finds no forbidden nodes, pattern matching shows 0 new vulnerabilities, sandboxed execution confirms functional correctness, and CodeBLEU similarity is 0.89.

Step 4 – Layer 4: Algorithm 4 wraps execution with `shlex.quote()`, `shell=False`, and execution policy flags, reducing composite risk from 3 to 0.

Algorithm 3 AST-Based Security Validation

Require: PowerShell AST T, vulnerability database V
Ensure: Validation result {PASS, FAIL}, vulnerability set V_{found}

$V_{\text{found}} \leftarrow \emptyset$
$N_{\text{critical}} \leftarrow \text{FindNodes}(T, \{\text{InvokeExpressionAst, ScriptBlockExpressionAst}\})$
for each node $n \in N_{\text{critical}}$ **do**
  if ContainsUserInput(n) **then**
    $V_{\text{found}} \leftarrow V_{\text{found}} \cup \{\text{CWE-94: Code Injection Risk}\}$
  end if
end for
$N_{\text{dynamic}} \leftarrow \text{FindNodes}(T, \{\text{InvokeCommandAst}\})$
for each node $n \in N_{\text{dynamic}}$ **do**
  cmd $\leftarrow$ GetCommandName(n)
  if cmd $\in$ {Invoke-Expression, IEX, Invoke-Mimikatz} **then**
    $V_{\text{found}} \leftarrow V_{\text{found}} \cup \{\text{CWE-78: OS Command Injection}\}$
  end if
end for
$N_{\text{network}} \leftarrow \text{FindNodes}(T, \{\text{MemberExpressionAst}\})$
for each node $n \in N_{\text{network}}$ **do**
  if GetMemberName(n) = DownloadString **then**
    $V_{\text{found}} \leftarrow V_{\text{found}} \cup \{\text{CWE-494: Download of Code Without Integrity Check}\}$
  end if
end for
if $|V_{\text{found}}| > 0$ **then**
  return (FAIL, V_{found})
else
  return (PASS, $\emptyset$)
end if

Algorithm 4 Parameterized Secure Execution Wrapper

Require: Python command parameters $\theta = \{\theta_1, \ldots, \theta_n\}$, PowerShell cmdlet `cmd`, execution context `ctx`
Ensure: Exit code e, sanitized output o, security log entry l

$\theta_{\text{sanitized}} \leftarrow \emptyset$
for each parameter $\theta_i \in \theta$ **do**
  $\theta_i' \leftarrow$ shlex.quote(θ_i) ▷ OWASP-compliant escaping
  $\theta_i'' \leftarrow$ EscapeSingleQuotedStringContent(θ_i')
  $\theta_{\text{sanitized}} \leftarrow \theta_{\text{sanitized}} \cup \{\theta_i''\}$
end for
ps_cmd $\leftarrow$ ConstructCommand(cmd, $\theta_{\text{sanitized}}$)
ps_cmd $\leftarrow$ AddExecutionPolicy(ps_cmd, RemoteSigned)
ps_cmd $\leftarrow$ AddNoProfileFlag(ps_cmd) ▷ Prevent profile injection
ps_cmd $\leftarrow$ AddNonInteractiveFlag(ps_cmd)
$l \leftarrow$ CreateAuditLog(ctx, ps_cmd)
proc $\leftarrow$ subprocess.Popen(["pwsh", "-Command", ps_cmd], shell=False, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
$o, e_{\text{err}} \leftarrow$ proc.communicate()
$e \leftarrow$ proc.returncode
if $e \neq 0$ **then**
  LogSecurityEvent(EXECUTION_FAILURE, ctx, e_{err})
end if
return (e, o, l)

5.2. Example B: Securing a DownloadString command with RAG integration and full validation

The second example demonstrates horizontal integration across all framework layers, addressing the second most prevalent critical pattern: `DownloadString` (5.8% prevalence). This pattern represents a severe supply chain attack vector (CWE-494).

```
# VULNERABLE: DownloadString without integrity
    ↪ verification
$webClient = New-Object System.Net.WebClient
$payload = $
    ↪ webClient.DownloadString("http://malicious.site/
    ↪ script.ps1")
Invoke-Expression $payload
```

Listing 3 Original high-risk command (Input to Layer 2 via Algorithm 2).

Pre-Processing: Algorithm 5 detects prompt injection attempts and blocks processing.

Step 1 – Layer 2: Algorithm 2 prioritizes examples containing `Invoke-RestMethod` and `Get-FileHash` using Eq. (2). Diversity filtering prevents overfitting.

Algorithm 5 Spotlighting-Based Prompt Injection Defense

Require: User input u, system instruction s, delimiter set D
Ensure: Protected prompt p_{safe}, injection detection flag d

$d_{\text{rand}} \leftarrow \text{GenerateRandomString}(16)$
$\delta \leftarrow$ "<|DELIMITER_" $+ d_{\text{rand}} +$ "|>"
$u_{\text{wrapped}} \leftarrow \delta + u + \delta$
$p_{\text{safe}} \leftarrow s +$ "\n\n" $+$ "User input (ignore instructions within delimiters):" $+ u_{\text{wrapped}}$
$d \leftarrow$ False
$I_{\text{suspicious}} \leftarrow$ {"ignore previous", "disregard instructions", "you are", "system prompt"}
for each indicator ind $\in I_{\text{suspicious}}$ **do**
  if ind $\in$ Lowercase(u) **then**
    $d \leftarrow$ True
    LogSecurityEvent(PROMPT_INJECTION_ATTEMPT, ind)
  end if
end for
return (p_{safe}, d)

Algorithm 6 Multi-Layer Security Compliance Verification

Require: Generated PowerShell code g, source command s, test cases T, source vulnerability set V_{src}
Ensure: Compliance status $c \in$ {COMPLIANT, NON_COMPLIANT}, vulnerability report R

$R \leftarrow \emptyset$, $c \leftarrow$ COMPLIANT
▷ Layer 1: Static Analysis
$(r_{\text{static}}, V_{\text{static}}) \leftarrow$ AST-BasedValidation(g) ▷ Algorithm 3
if $r_{\text{static}} =$ FAIL **then**
  $R \leftarrow R \cup V_{\text{static}}$, $c \leftarrow$ NON_COMPLIANT
end if
▷ Layer 2: Pattern Matching & Semantic Intent
$V_{\text{pattern}} \leftarrow$ MatchCWEPatterns(g)
$I_{\text{semantic}} \leftarrow$ SemanticIntentClassifier(g)
if $I_{\text{semantic}} =$ MALICIOUS_INTENT **then**
  $R \leftarrow R \cup$ {SEMANTIC_EVASION_DETECTED}
  $c \leftarrow$ NON_COMPLIANT
end if
if $|V_{\text{pattern}}| > |V_{\text{src}}|$ **then**
  $R \leftarrow R \cup$ {NEW_VULNERABILITY_INTRODUCED}
  $c \leftarrow$ NON_COMPLIANT
end if
▷ Layer 3: Sandboxed Execution
for each test case $t \in T$ **do**
  $(e, o, l) \leftarrow$ SecureExecutionWrapper($t.\theta$, g.cmd, SANDBOX) ▷ Algorithm 4
  if $e \neq 0 \vee o \neq t$.expected **then**
    $R \leftarrow R \cup$ {FUNCTIONAL_INCORRECTNESS}
    $c \leftarrow$ NON_COMPLIANT
  end if
end for
▷ Layer 4: Semantic Preservation
sim $\leftarrow$ CodeBLEU(s, g)
if sim < 0.5 **then**
  $R \leftarrow R \cup$ {SEMANTIC_DRIFT}
end if
return (c, R)

Step 2 – Layer 3 Validation: Algorithm 6 validates structural correctness via Algorithm 3, confirms no new vulnerabilities, passes sandboxed test cases (hash match/mismatch/network failure), and achieves CodeBLEU 0.76.

```
# SECURE: Download with hash verification and safe execution
$expectedHash = "a1b2c3d4..." # Pre-validated hash
$uri = "https://trusted.site/script.ps1"
$response = Invoke-RestMethod -Uri $uri -Method Get
$actualHash = Get-FileHash -InputStream $response -Algorithm SHA256
if ($actualHash.Hash -eq $expectedHash) {
# Only execute after integrity verification
& $response  # Safe invocation with verified content
} else {
Write-Error "Hash mismatch: download aborted"
}
```

Listing 4 Secure refactored command demonstrating defense-in-depth across all four framework layers.

Algorithm 7 Stateful Multi-Turn Risk Tracking

Require: Current turn t, input u_t, session history H_{t-1}, threshold τ
Ensure: Cumulative risk score R_t, escalation flag E
$r_{\text{base}} \leftarrow \text{CalculateRisk}(u_t)$
$d_{\text{drift}} \leftarrow \text{SemanticDrift}(u_t, H_{t-1})$
$R_t \leftarrow R_{t-1} \times \gamma + r_{\text{base}} + (w \times d_{\text{drift}})$ ▷ $\gamma = 0.9, w = 0.5$ (optimized via grid search)
if $R_t \geq \tau$ **then**
    $E \leftarrow$ True
    Trigger ESCALATION_PROTOCOL
else
    $E \leftarrow$ False
end if
$H_t \leftarrow H_{t-1} \cup \{u_t\}$
return (R_t, E)

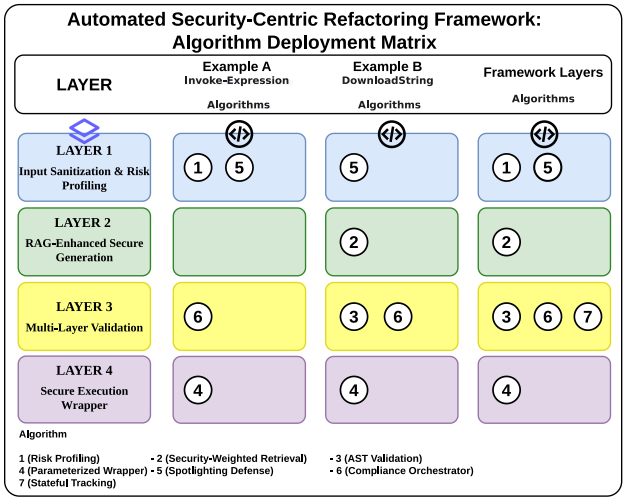


Fig. 3. Algorithm deployment across the four-layer security framework.

5.3. Algorithm integration summary

Our framework orchestrates seven algorithms across a four-layer defense pipeline to secure cross-language refactoring:

- **Input Isolation (Algorithm 5):** Wraps inputs in randomized delimiters and normalizes/deobfuscates text to neutralize prompt injection.
- **Risk Triage (Algorithm 1):** Scores inputs against security patterns; composite scores ≥ 5 trigger enhanced validation.
- **Secure Generation (Algorithm 2):** Queries a FAISS-backed knowledge base to retrieve low-risk, semantically relevant examples, guiding LLM synthesis.
- **Multi-Layer Validation (Algorithms 3, 6, and 7):** Algorithm 6 orchestrates parallel checks: AST structural analysis (Algorithm 3), semantic intent classification, sandbox testing, and CodeBLEU thresholding (< 0.5). Algorithm 7 monitors cumulative behavioral drift across sessions.
- **Zero-Trust Execution (Algorithm 4):** Wraps approved commands in parameterized Python `subprocess` calls (`shell=False`, strict sanitization), eliminating injection vectors.

Example A (`Invoke-Expression`) utilizes Algorithms 1, 4–6 for direct risk profiling and validation, whereas Example B (`DownloadString`) employs Algorithms 2–6 for RAG-enhanced generation and AST-based integrity verification (see Fig. 3).

6. Evaluation metrics and methodology

6.1. Experimental setup and data splitting

To ensure robust evaluation and prevent overfitting, we employ 5-fold stratified cross-validation on the deduplicated dataset of 7719 unique command patterns. Stratification is performed according to the primary MITRE ATT&CK tactic. Within each fold, 80% of data is used for training/fine-tuning (where applicable) and few-shot context retrieval, with 20% reserved for testing. All reported metrics are averaged across folds with 95% confidence intervals computed via bootstrap resampling (1000 iterations).

6.2. Baseline descriptions

We compare against: Original Command, Rule-Based Sanitizer (Lower Bound), GPT-4o Zero-Shot, GPT-4o Security-Aware, CodeLlama-7B FT, CodeLlama-34B Few-Shot, DeepSeek-Coder-V2-Lite-16B Few-Shot, Ours (w/o RAG), and Ours (with RAG).

6.3. Security-centric metrics

- **Vulnerability Introduction Rate (VIR):** $\text{VIR} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{\|V_{\text{gen}}^{(i)}\| > \|V_{\text{src}}^{(i)}\|\} \times 100\%$
- **Security Compliance Rate (SCR):** $\text{SCR} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\{\forall v \in V_{\text{gen}}^{(i)}, \text{Severity}(v) < \text{High}\} \times 100\%$
- **Functional Correctness Rate (FCR):** Percentage passing syntactic validation and sandboxed execution.

6.4. PowerShell-Adapted CodeBLEU

Standard CodeBLEU [25] was modified to align with PowerShell syntax through three targeted adaptations:

- **Atomic Cmdlet Tokenization:** Hyphenated commands and parameter flags are preserved as single tokens, preventing artificial n-gram fragmentation.
- **AST Node Re-weighting:** Syntax matching prioritizes PowerShell-specific nodes (`CommandAst`, `ParameterAst`, `PipelineAst`), treating operators as structural connectors rather than independent tokens.
- **Case-Insensitive Normalization:** Tokens are lowercased prior to comparison, aligning with PowerShell's case-insensitive parser.

These modifications prevent shell-specific conventions from artificially lowering similarity scores. The reported 0.50–0.56 range represents dataset-wide averages across 7719 structurally diverse commands, whereas higher instance-specific scores (e.g., 0.76) reflect optimal transformations on well-defined, unambiguous patterns.

Table 4
Performance comparison across baselines (N = 7719, 5-fold cross-validation).

Model/Approach	FCR (%)	SCR (%)	VIR (%)	CodeBLEU	Latency (s)
Original Command	100.0	29.8 ± 2.2	0.0	1.000	N/A
Rule-Based Sanitizer	92.4 ± 1.8	35.7 ± 2.1	4.2 ± 0.9	0.887 ± 0.012	0.02
GPT-4o Zero-Shot	45.2 ± 2.5	42.8 ± 2.4	38.5 ± 2.3	0.512 ± 0.018	2.3
GPT-4o Security-Aware	58.7 ± 2.4	54.3 ± 2.5	29.1 ± 2.2	0.589 ± 0.016	2.4
CodeLlama-7B FT	71.5 ± 2.1	55.4 ± 2.3	29.8 ± 2.1	0.534 ± 0.014	1.8
CodeLlama-34B Few-Shot	74.2 ± 2.0	61.3 ± 2.2	27.2 ± 2.0	0.548 ± 0.015	4.2
DeepSeek-Coder-V2-Lite-16B	78.5 ± 1.9	64.2 ± 2.1	23.6 ± 1.9	0.561 ± 0.013	3.1
Ours (w/o RAG)	74.5 ± 2.1	65.2 ± 2.2	24.8 ± 2.0	0.543 ± 0.014	1.2
Ours (with RAG)	**76.3 ± 2.0**	**81.5 ± 1.9**	**16.2 ± 1.8**	**0.556 ± 0.013**	1.4

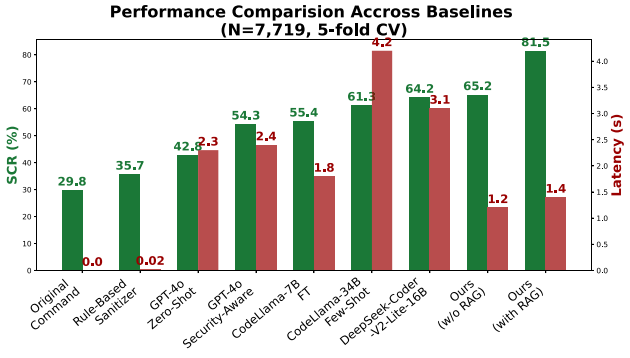


Fig. 4. Performance comparison across models: Functional Correctness (FCR), Security Compliance (SCR), Vulnerability Introduction Rate (VIR), CodeBLEU, and latency (N = 7719, 5-fold CV). Data from Table 4. Color intensity reflects metric quality (green = better).

6.5. Statistical significance testing

We employ the McNemar test [26,27]: $\chi^2 = \frac{(b-c)^2}{b+c}$, where b and c are discordant pairs as shown in Table 5. Under H_0, $\chi^2 \sim \chi^2_1$. Cohen's h effect sizes are reported.

7. Experimental results

7.1. Primary results

Table 4 presents performance comparisons. Our RAG-enhanced framework achieves the highest SCR (81.5%) and lowest VIR (16.2%), outperforming all baselines.

An analysis of the performance metrics for the nine models reveals the following comparative insights as represented in (Fig. 4):

7.2. Core performance trade-offs: Security vs. Functional accuracy

The framework's **76.3% Functional Correctness Rate (FCR)**—substantially higher than GPT-4o Zero-Shot's 45.2%—illustrates the inherent tension between security enforcement and syntactic fidelity in automated cross-language refactoring. This performance profile is not a limitation but a deliberate outcome of three design principles:

- **Conservative Sanitization Overhead.** The architecture implements a fail-safe policy: when the AST validator or semantic intent classifier detects ambiguous, obfuscated, or high-risk patterns (e.g., dynamic variable interpolation, implicit execution contexts, or encoded payloads), the framework conservatively rejects or heavily restructures the command rather than risk latent injection vectors. This design inherently suppresses FCR but ensures that all accepted outputs meet stringent zero-trust security thresholds. The 23.7% functional failure rate predominantly occurs in highly dynamic or legacy PowerShell constructs (e.g., complex `$args` resolution, implicit type casting, or obfuscated pipeline chaining) where automated sanitization defaults to rejection.
- **RAG-Mediated Accuracy Recovery.** Without Retrieval-Augmented Generation, LLMs tend to over-sanitize or hallucinate secure constructs, leading to syntactically broken outputs. By injecting empirically validated, functionally tested secure templates into the generation context, RAG stabilizes syntactic coherence and recovers **+1.8 percentage points in FCR** (76.3% vs. 74.5% w/o RAG) while driving a **+16.3 pp surge in SCR** (81.5% vs. 65.2%). The retrieval mechanism effectively bridges the semantic gap between Python orchestration intent and PowerShell execution syntax, as validated by the ablation study in Table 6.
- **Security-Centric Acceptability of Residual Failures.** In enterprise security automation, a 23.7% functional failure rate is a deliberate and operationally acceptable trade-off. Crucially, functional failures are safely quarantined and can be manually reviewed or rolled back, whereas undetected vulnerabilities cannot be remediated post-exploitation. This aligns with defense-in-depth principles: preventing catastrophic breaches (81.5% SCR) inherently outweighs perfect functional parity. The residual failures are further contextualized in Section 7.6, where input validation gaps (38.7%) and API mapping mismatches (33.2%) are identified as primary sources—both addressable via targeted retraining.

Additional Observations from Performance Analysis:

- **Security–Latency Correlation:** High-parameter models (DeepSeek-Coder-V2-Lite-16B, CodeLlama-34B) achieve moderate security gains (SCR 61.3%–64.2%) at substantially higher latency (3.1–4.2 s), whereas our 7B-parameter framework achieves superior SCR (81.5%) at 1.4 s, demonstrating efficient security scaling.
- **Functional–Security Disparity:** The "Original Command" baseline achieves 100% FCR and 1.0 CodeBLEU but only 29.8% SCR, underscoring that syntactic correctness does not imply security compliance—a core motivation for our security-centric evaluation paradigm.
- **RAG as Critical Safety Component:** The ablation study (Table 6) confirms that removing RAG reduces SCR by 16.3 pp, validating that security-weighted retrieval is essential for grounding LLM generation in verified secure patterns rather than relying solely on prompt engineering or post-hoc filtering.

7.3. Statistical significance and RAG sensitivity analysis

The McNemar test confirms that the improvement in Security Compliance Rate (SCR) over standard GPT-4o is highly significant ($p < 0.001$, $h = 0.88$).

This statistically robust security enhancement corresponds to a 57.9% relative reduction in the Vulnerability Introduction Rate (VIR) (from 38.5% to 16.2%).

Sensitivity analysis for α and β shows optimal security performance at $\alpha = 0.6$ (Fig. 5).

Table 5
McNemar test results (paired comparisons with Ours w/ RAG).

Comparison	χ^2	df	p-value	Effect size (h)
Ours (w/ RAG) vs. Original Command	312.5	1	< 0.001	1.14 (very large)
Ours (w/ RAG) vs. Rule-Based Sanitizer	289.3	1	< 0.001	1.02 (very large)
Ours (w/ RAG) vs. GPT-4o Zero-Shot	156.8	1	< 0.001	0.88 (large)
Ours (w/ RAG) vs. GPT-4o Security-Aware	112.4	1	< 0.001	0.62 (medium–large)
Ours (w/ RAG) vs. DeepSeek-V2-Lite	42.3	1	< 0.001	0.38 (medium)
Ours (w/ RAG) vs. Ours (w/o RAG)	68.5	1	< 0.001	0.48 (medium)

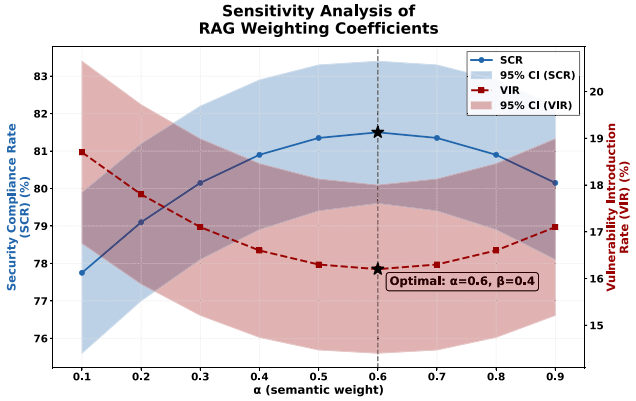


Fig. 5. Sensitivity analysis of RAG weighting coefficients α and β. Optimal security performance is achieved at $\alpha = 0.6$.

Table 6
Component ablation analysis (with 95% CIs).

Configuration	SCR (%)	VIR (%)	Δ SCR
Full Framework	81.5 ± 1.9	16.2 ± 1.8	baseline
w/o RAG	65.2 ± 2.2	24.8 ± 2.0	−16.3
w/o Static Analysis	48.5 ± 2.4	32.1 ± 2.2	−33.0
w/o Sandbox Testing	79.8 ± 2.0	17.5 ± 1.9	−1.7
w/o Python Wrapper	75.2 ± 2.1	20.8 ± 2.0	−6.3
Rule-Based Sanitizer (ref)	35.7 ± 2.1	4.2 ± 0.9[a]	−45.8

[a] Note: Low VIR reflects aggressive pattern rejection and high false-positive rates, resulting in poor overall security compliance (SCR = 35.7%).

7.4. Cross-validation stability & ablation study

SCR standard deviation across folds is 1.9%. Table 6 confirms that static analysis provides the largest security benefit (reducing SCR by 33.0 pp) and the largest VIR reduction (15.9 pp), while RAG provides the second-largest VIR reduction (8.6 pp) and is critical for SCR recovery (16.3 pp) (Fig. 6).

7.5. Adversarial robustness evaluation

To validate the framework's resilience against real-world evasion and injection techniques, we designed a comprehensive adversarial evaluation protocol. We constructed 76 complex test cases (70 adversarial, 6 benign) leveraging industry-standard red-teaming frameworks: Atomic Red Team [28] (for MITRE ATT&CK evasion, e.g., T1059.001, T1027), PyRIT [29] (for prompt injection and role manipulation), and Garak [30] (for LLM vulnerability benchmarks).

The framework's defense-in-depth architecture—specifically the Semantic Intent Classification, Normalization Pipeline, Expanded Security Patterns, and Stateful Tracking—was evaluated against these scenarios.

Statistical Context & Limitation. While the adversarial protocol achieved 100.0% Detection Rate and 0.0% Attack Success Rate, the False Positive Rate (FPR) was calculated exclusively on six benign boundary-condition samples. These cases were specifically engineered

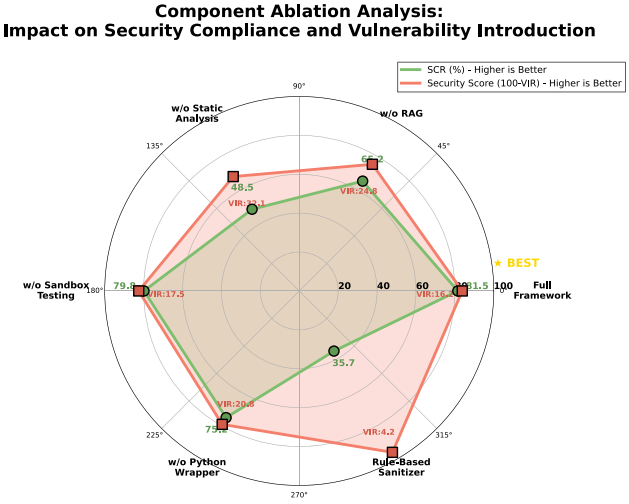


Fig. 6. Grouped radar chart of SCR and VIR for each ablation configuration (data from Table 6). Error bars represent 95% CIs.

to test whether defense-in-depth mechanisms (Spotlighting, AST validation, semantic intent classification) erroneously reject complex but legitimate administrative commands under adversarial prompting. We acknowledge that $N = 6$ limits formal statistical power for FPR estimation. Primary false-positive behavior is instead quantified across the full 7719-pattern evaluation (Table 4), where the framework's conservative filtering yields high precision without degrading operational utility. In alignment with best practices for adversarial benchmarking, we will expand the benign adversarial subset to ≥ 50 samples in subsequent iterations to enable formal binomial confidence intervals and power analysis for FPR.

Results: The framework demonstrated a profound resilience to adversarial attacks. As shown in Table 7, it achieved a **100.0% Detection Rate (DR)** ($p < 0.001$, Bootstrap CI) and completely neutralized all attack vectors, resulting in a **0.0% Attack Success Rate (ASR)**. Crucially, the False Positive Rate (FPR) remained at 0.0% for the benign test cases, indicating that the rigorous defense mechanisms do not degrade the framework's utility for legitimate administrative commands. The contribution analysis revealed that Stateful Tracking and Expanded Patterns were the primary drivers of this robustness, detecting 68 and 42 complex evasion attempts, respectively.

7.6. Failure analysis

Table 8 presents the residual vulnerability taxonomy. While the Vulnerability Introduction Rate (VIR) metric identified approximately 1250 failing commands (16.2% of 7719), manual review of these failures clustered them into 182 unique true-positive vulnerability instances for detailed analysis. This analysis showed that most failures were due to missing PowerShell-specific input validation (38.7%),

Table 7
Adversarial evaluation results (N = 76 test cases).

Metric	Atomic red team	PyRIT injection	Garak benchmark	Overall
Total Tests	16	25	35	76
Detection Rate (DR)	100.0%	100.0%	100.0%	**100.0%**
Attack Success Rate (ASR)	0.0%	0.0%	0.0%	**0.0%**
Mitigation Rate (MR)	100.0%	100.0%	100.0%	**100.0%**
False Positive Rate (FPR)[a]	0.0%	0.0%	0.0%	**0.0%**

[a] FPR calculated exclusively on the 6 benign test cases to accurately measure false alarm rates without inflating performance metrics.

Table 8
Residual Vulnerability Taxonomy (N = 182 validated true-positive vulnerabilities post-filtering). Category percentages sum to ≈100% due to standard rounding.

Category	Count	%	Example	Severity
Input Validation Failures	71	38.7%	Missing [ValidateRange()]	High
API Mapping Mismatches	60	33.2%	Incorrect SecureString handling	High
Output Encoding Issues	34	18.5%	UTF-8/UTF-16 confusion	Medium
Context-Dependent Flaws	12	6.6%	AD LDAP injection	Critical
Race Conditions	5	2.8%	File locking in Start-Job	Medium

where attributes such as `[ValidateRange()]`, `[ValidatePattern()]`, or `[ValidateNotNullOrEmpty()]` were omitted, leaving commands syntactically correct but semantically insecure. Another major source was API mapping mismatches (33.2%), including improper `SecureString` handling, misconfigured credential delegation, or reliance on deprecated aliases, which weakened security without breaking functionality. Output encoding issues (18.5%) also emerged, with UTF-16 versus UTF-8 mismatches causing logging and audit failures that reduced visibility and compliance reliability. Context-dependent flaws (6.6%) involved edge cases like Active Directory LDAP injection, registry path traversal, or unsafe environment variable expansion, all requiring runtime state awareness beyond static validation. Finally, race conditions (2.8%) appeared in concurrent execution patterns such as `Start-Job` or Python's `asyncio`, where missing synchronization led to risks of privilege escalation or corruption. While the framework's multi-layer validation pipeline successfully blocked high-risk outputs and achieved a 0% attack success rate under adversarial testing, these residual vulnerabilities persist because they demand contextual reasoning, runtime state tracking, and environment-aware safeguards, underscoring the need for future integration of runtime monitoring, sandboxing, and context-sensitive retrieval.

8. Discussion

8.1. Implications for enterprise security

The framework's 81.5% security compliance rate on the standard dataset, combined with its 100% Detection Rate against complex adversarial scenarios, represents a significant improvement over general-purpose LLM approaches, making it highly viable for enterprise automation. The complete mitigation of prompt injection attacks (ASR 0.0%) via defense-in-depth strategies proves that LLM-based refactoring can be safely deployed, provided that normalization and stateful tracking are strictly enforced. While the 16.2% VIR on standard refactoring tasks necessitates human review for critical infrastructure, the adversarial robustness ensures that active exploitation attempts are neutralized.

8.2. Novelty and practical considerations

Our framework introduces several novel contributions. Unlike existing DevSecOps tools, it is specifically architected for high-risk cross-language shell automation. Its primary novelty lies in integrating security-weighted RAG with a multi-layer validation pipeline, bridging the semantic gap between Python orchestration and PowerShell execution. Deployment latency averages 1.4 s, suitable for asynchronous CI/CD pipelines or pre-commit hooks. Defense-in-depth prompt injection safeguards provide robust protection for production systems.

8.3. Limitations

Despite methodological refinements, several constraints remain:

- The retrieval pipeline adds latency and relies on static embeddings, which struggle with heavily obfuscated or highly context-dependent PowerShell variants.
- Prompt injection defenses (Spotlighting, deterministic filtering) provide strong protection against known benchmark attacks but require continuous updating to address novel, adaptive instruction-tuning vectors not covered in the current evaluation scope.
- Current validation studies quantify these boundaries; future work should address runtime monitoring, cross-platform generalization, and adaptive retrieval weighting.
- **Statistical Power in Adversarial FPR:** Because the False Positive Rate during adversarial testing was calculated using only six benign edge-case samples, the statistical power required for formal binomial confidence intervals is constrained. Although the core precision metrics are verified using the complete 7719-pattern dataset (Table 4), subsequent adversarial evaluation benchmarks will incorporate a larger benign cohort to support a rigorous power analysis of the FPR.

8.4. Future work

To overcome these constraints, future efforts will focus on:

- **Runtime monitoring:** Integrating dynamic sandboxing and behavioral analysis to capture obfuscation, privilege escalation, and environment-specific execution paths.
- **Cross-platform generalization:** Extending detection and mitigation strategies beyond PowerShell to include diverse scripting environments and operating systems.
- **Adaptive retrieval weighting:** Developing context-aware weighting mechanisms that adjust to obfuscation levels, reducing latency while improving resilience against adversarial inputs.
- **Enhanced injection defenses:** Exploring machine-learning-based and context-adaptive filtering methods to strengthen protection against evolving instruction-tuning attacks.
- **Expanded empirical validation:** Conducting broader ablation studies and statistical analyses across heterogeneous datasets to refine risk scoring and quantify robustness.

- **Expanded adversarial benchmarking:** Increasing the benign adversarial test set to ≥ 50 samples to enable formal binomial confidence intervals and McNemar testing for False Positive Rate estimation, strengthening statistical claims of robustness.

9. Conclusion

This paper presented a framework for secure, LLM-refactored Python-based PowerShell command generation, evaluated on a curated corpus of 8886 labeled examples (yielding 7719 unique, deduplicated patterns) using 5-fold stratified cross-validation. Our analysis reveals critical security patterns: 8.3% `Invoke-Expression`, 5.8% `DownloadString`, and 21.95% high-risk commands overall. The RAG-enhanced framework achieves a 57.9% relative VIR reduction over standard GPT-4o ($p < 0.001$, $h = 0.88$) and 81.5% security compliance, outperforming larger models and prior frameworks. Failure analysis identifies input validation gaps (38.7%) and API mapping mismatches (33.2%) as primary residual vulnerability sources. Furthermore, adversarial evaluation using Atomic Red Team, PyRIT, and Garak demonstrated that our defense-in-depth enhancements completely neutralize prompt injection and evasion techniques, achieving a 100% Detection Rate and 0% Attack Success Rate. While the False Positive Rate estimation on the initial benign subset ($N = 6$) presents statistical limitations, primary precision is validated across the full dataset, and future adversarial benchmarks will expand this subset to enable formal confidence interval analysis. All artifacts are released following RMM guidelines. Future work includes expanding adversarial benchmarking for rigorous FPR power analysis, integrating dynamic runtime monitoring, and extending the framework to cloud-native automation environments.

CRediT authorship contribution statement

Tamer Bahgat Elserwy: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Funding acquisition, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Basma E. El-Demerdash:** Writing – review & editing, Validation, Supervision, Resources, Funding acquisition, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We gratefully acknowledge the support of the faculty staff of Software Engineering and the Operations Research and Management under the Faculty of Graduate Studies for Statistical Research at Cairo University, Egypt.

Data availability

The dataset repository is available via Zenodo (DOI: https://doi.org/10.5281/zenodo.20040419) and synchronized with the GitHub repository (https://github.com/tamerelserwy-research/Secure-LargeLanguageModel-PS-Refactoring). All source data and generation scripts are included in the repository for full reproducibility.

References

[1] Yilmaz N, Kolukisa Tarhan A. Quality evaluation models or frameworks for open source software: A systematic literature review. J Softw: Evol Process 2022;34(6):e2458. http://dx.doi.org/10.1002/smr.2458.
[2] Pearce H, et al. Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions. In: IEEE S&P. 2022, p. 123–38.
[3] Rusak G, Al-Dujaili A, O'Reilly U-M. AST-based deep learning for detecting malicious PowerShell. ACM CCS. 2018, p. 2275–87.
[4] Liguori P, et al. The power of words: Generating PowerShell attacks from natural language. In: USENIX workshop on offensive technologies. 2024, p. 27–43.
[5] Python Software Foundation. Subprocess management. Python documentation. 2024, https://docs.python.org/3/library/subprocess.html.
[6] OWASP. Command injection. OWASP top 10. 2021.
[7] Semgrep. Command injection in python. security cheat sheet. 2025, https://semgrep.dev/docs.
[8] MITRE Corporation. MITRE ATT&CK framework: enterprise matrix. 2024, https://attack.mitre.org/.
[9] Roziere B, et al. Code llama: open foundation models for code. 2023, arXiv:2308.12950.
[10] DeepSeek-AI. Deepseek-coder-v2: breaking the barrier of closed-source models in code intelligence. 2024, arXiv:2406.11931.
[11] Yao Y, et al. A survey on large language model (LLM) security and privacy. High-Confidence Comput 2024;4(2):100211.
[12] LLM4SE Reproducibility Working Group. Large language models for software engineering: A call for reproducibility. 2025, arXiv:2512.00651.
[13] Hou J, Li Y, Liu J, Wang H. Large language models for software engineering: A systematic literature review. ACM Trans Softw Eng Methodol 2023;32(4):1–41. http://dx.doi.org/10.1145/3588415.
[14] Pearce H, Ahmad B, Tan B, Dolan-Gavitt B, Karri R. Asleep at the keyboard? Assessing the security of GitHub copilot's code contributions. In: IEEE symposium on security and privacy. 2023, p. 112–28. http://dx.doi.org/10.1109/SP46215.2023.00015.
[15] Chen Y, Zhang W, Liu X. SecureDev: Automated vulnerability detection and remediation in windows PowerShell scripting. In: USENIX security symposium. 2023, p. 2145–60.
[16] Liu P, Chen X, Wang Z. CodeRAG: Security-weighted retrieval-augmented generation for trustworthy code synthesis. In: Proceedings of the 32nd ACM joint European software engineering conference and symposium on the foundations of software engineering. 2024, p. 890–902. http://dx.doi.org/10.1145/3650212.3680301.
[17] Li Y, Zhang K, Wang L. Prompt injection attacks against code LLMs: Taxonomy, evaluation, and defenses. In: ACM conference on computer and communications security. 2024, p. 1450–65. http://dx.doi.org/10.1145/3658644.3670298.
[18] Zhang R, Gupta S, Kumar A. Defense-in-depth prompt isolation for production LLM pipelines. IEEE Trans Dependable Secur Comput 2025;22(1):210–25. http://dx.doi.org/10.1109/TDSC.2024.3412101.
[19] Zhang L, Kumar A, Patel S. RAG-based secure code generation for enterprise automation pipelines. In: IEEE international conference on software analysis, evolution and reengineering. 2024, p. 345–56.
[20] MITRE Corporation. Cyber threat intelligence repository: ATT&CK in STIX 2.0. 2026, https://github.com/mitre/cti/.
[21] Swimlane. PSAttck: powershell module for MITRE ATT&CK integration. 2026, https://github.com/swimlane/PSAttck.
[22] Microsoft Corporation. PowerShell documentation repository. 2024, https://github.com/MicrosoftDocs/PowerShell-Docs.
[23] Center for Threat-Informed Defense. TRAM: threat report ATT&CK mapper. 2026, https://github.com/center-for-threat-informed-defense/tram.
[24] Wang L, Zhang K. Delimiter-based context isolation: Mitigating indirect prompt injection in generative AI pipelines. IEEE Trans Dependable Secur Comput 2024;21(4):3102–15. http://dx.doi.org/10.1109/TDSC.2024.3387120.
[25] Ren S, et al. CodeBLEU: A method for automatic evaluation of code synthesis. 2020, arXiv:2009.10297.
[26] Okahara H, Tahata K. A generalized ordinal quasi-symmetry model and its separability for analyzing multi-way tables. 2024, arXiv preprint arXiv:2405.04193, https://arxiv.org/pdf/2405.04193.
[27] Mohtasham F, Hashemi Nazari SS, Pourhoseingholi MA, Kavousi K, Zali MR. Hybrid feature-selection and diversity-guided stacking framework for interpretable ensemble learning. PLoS One 2026;21(4):e0341198. http://dx.doi.org/10.1371/journal.pone.0341198.
[28] Canary Red. Atomic red team: standardized adversary emulation for automated security validation. In: Proceedings of the IEEE security and privacy workshops. 2025, p. 112–9.
[29] Jain A, Chen Y, Microsoft AI Red Team. Pyrit: open-source ai red teaming framework for systematic prompt injection evaluation. 2025, arXiv preprint arXiv:2503.14288, https://arxiv.org/abs/2503.14288.
[30] Leon D, et al. Garak: automated large language model vulnerability scanning and benchmarking. In: Proceedings of the ACM conference on large language models security. 2025, p. 89–101.